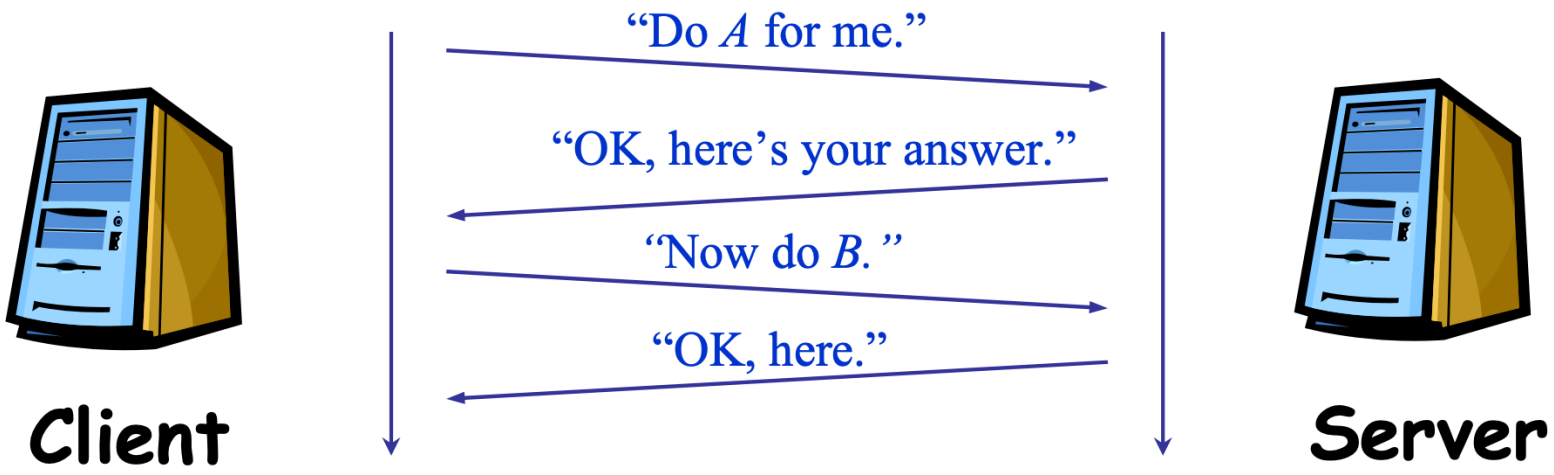
Network Programming
Peer-to-Peer

Minchen Yu
SDS@CUHK-SZ
Spring 2026

香港中文大學（深圳）
The Chinese University of Hong Kong, Shenzhen

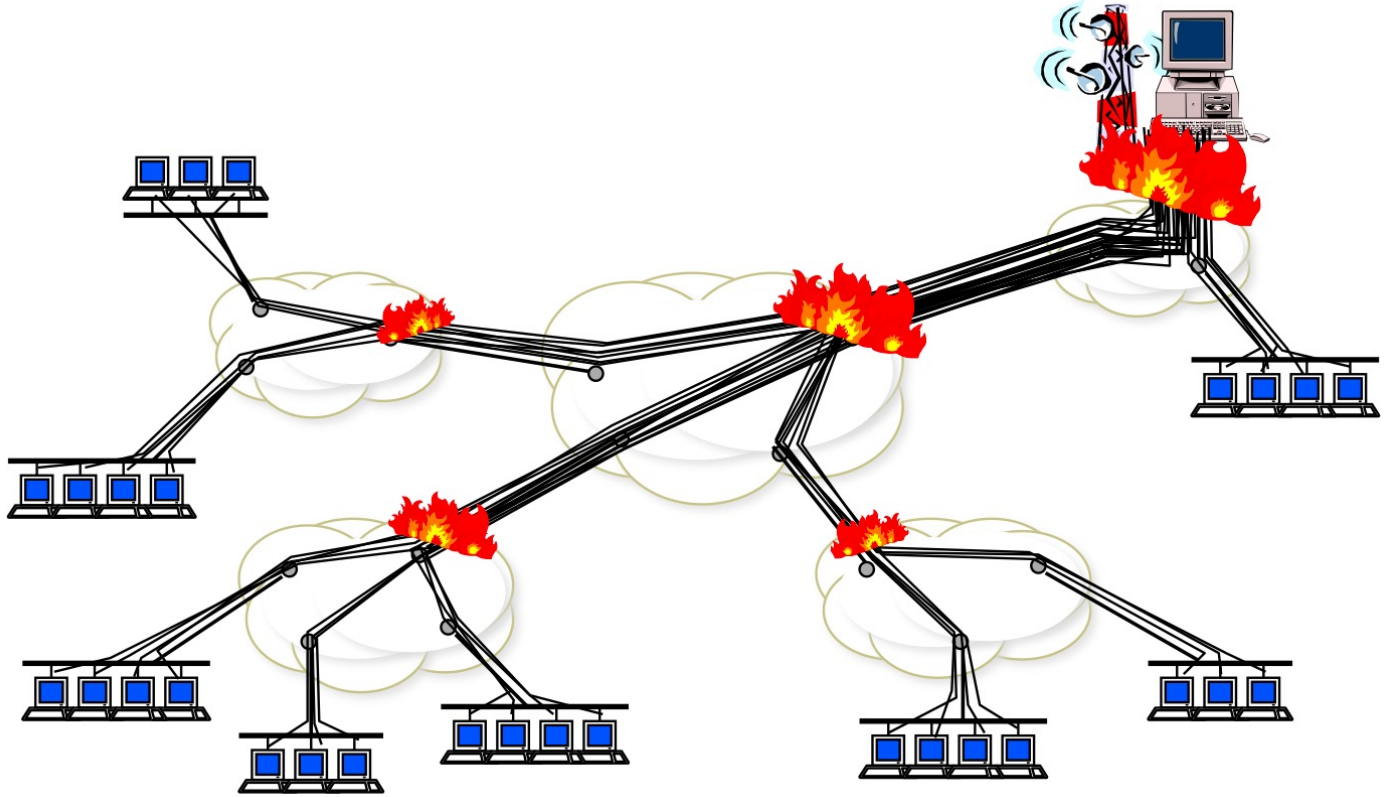
SCHOOL OF
DATA SCIENCE
數據科學學院

Recall Client/Server model



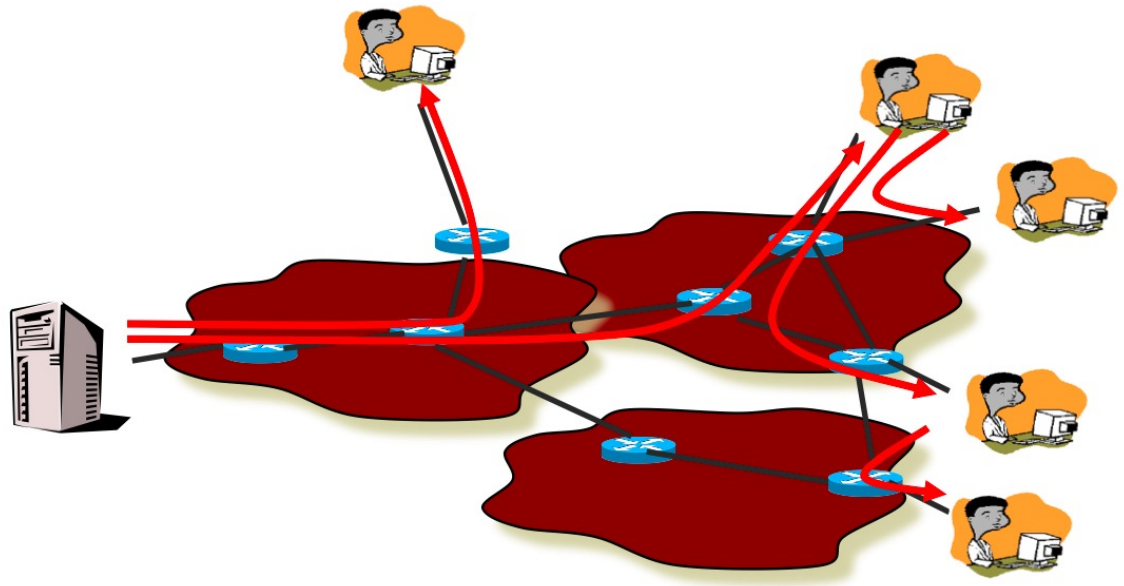
Scaling problem

- Millions of client -> Server and network meltdown



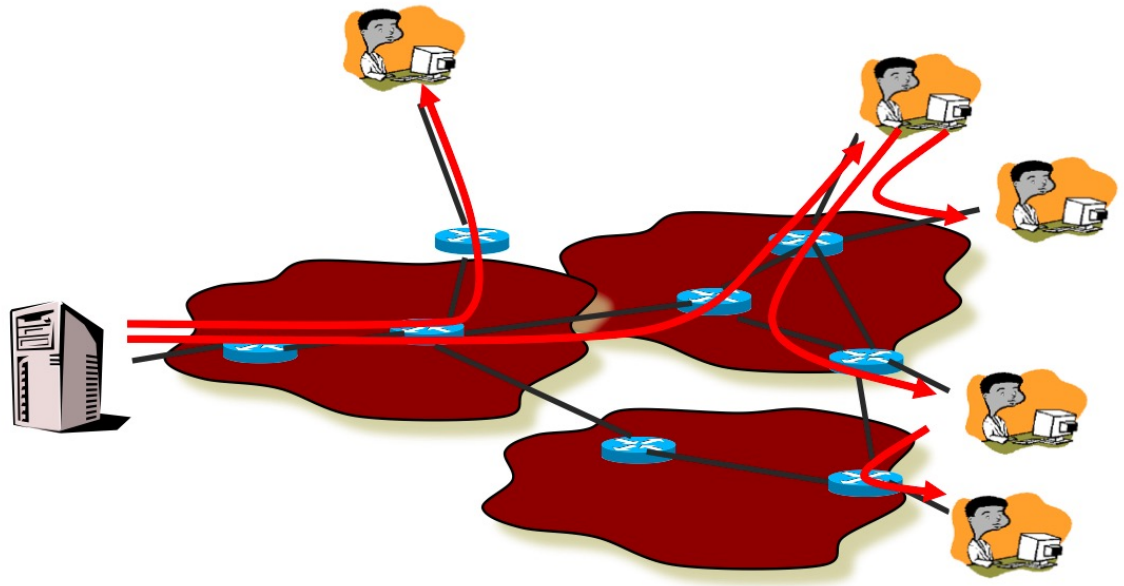
Solutions

- P2P systems
- Caching, DNS, CDNs (later)



P2P system

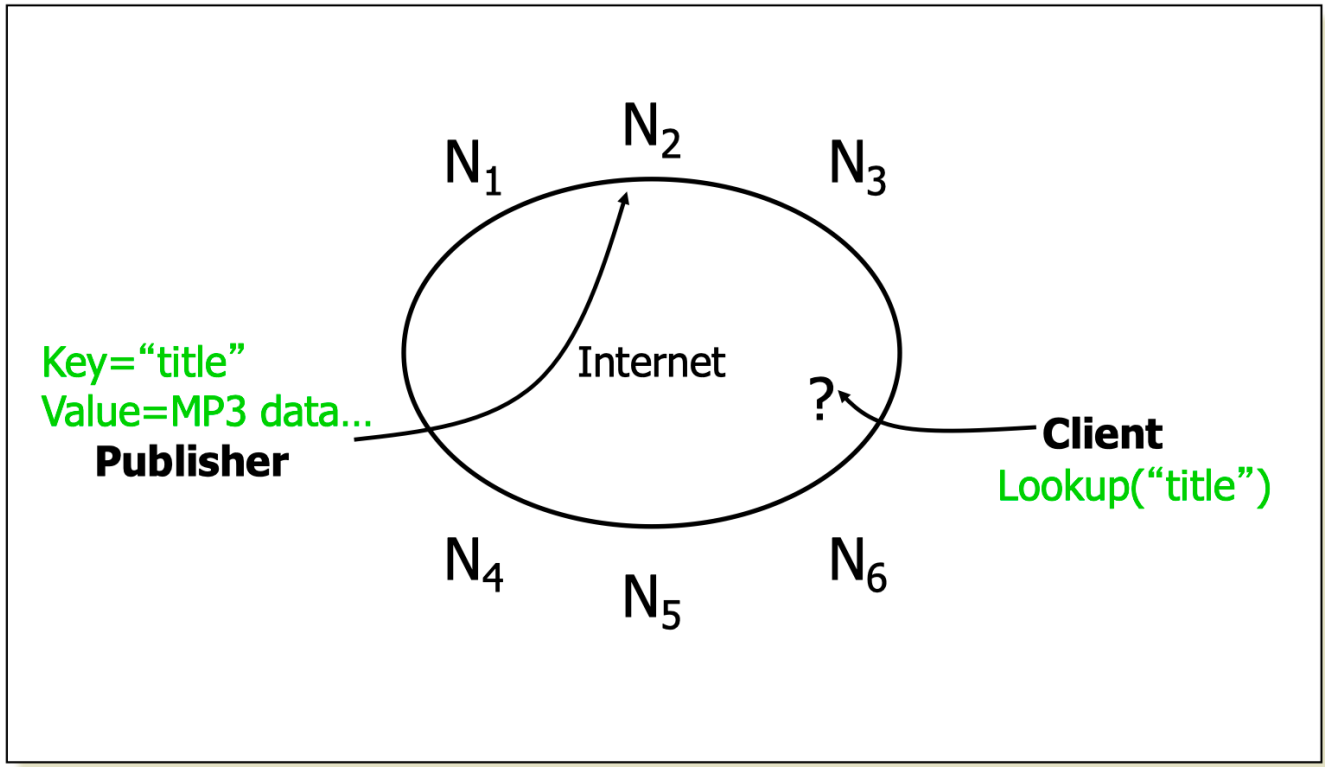
- Leverage the resources of client machines (peers)
  - Traditional: Computation, storage, bandwidth
  - Non-traditional: mobility, special token we call coins, sensors



P2P (storage) network

- Typically each member stores/provides access to content
- Basically a replication system for files
  - Always a tradeoff between possible location of files and searching difficulty
  - Peer-to-peer allow files to be anywhere -> searching is the challenge
  - Dynamic member list makes it more difficult: node churn

Lookup problem



Searching

- Needles vs. Haystacks
  - Searching for top 40, or an obscure punk track from 1981 that nobody's heard of?
- Search expressiveness
  - Whole word? Regular expressions? File names? Attributes? Whole-text search?
- Searching for recent versus older content
- Searching for content correlated with your location/time of day/etc versus not

Framework

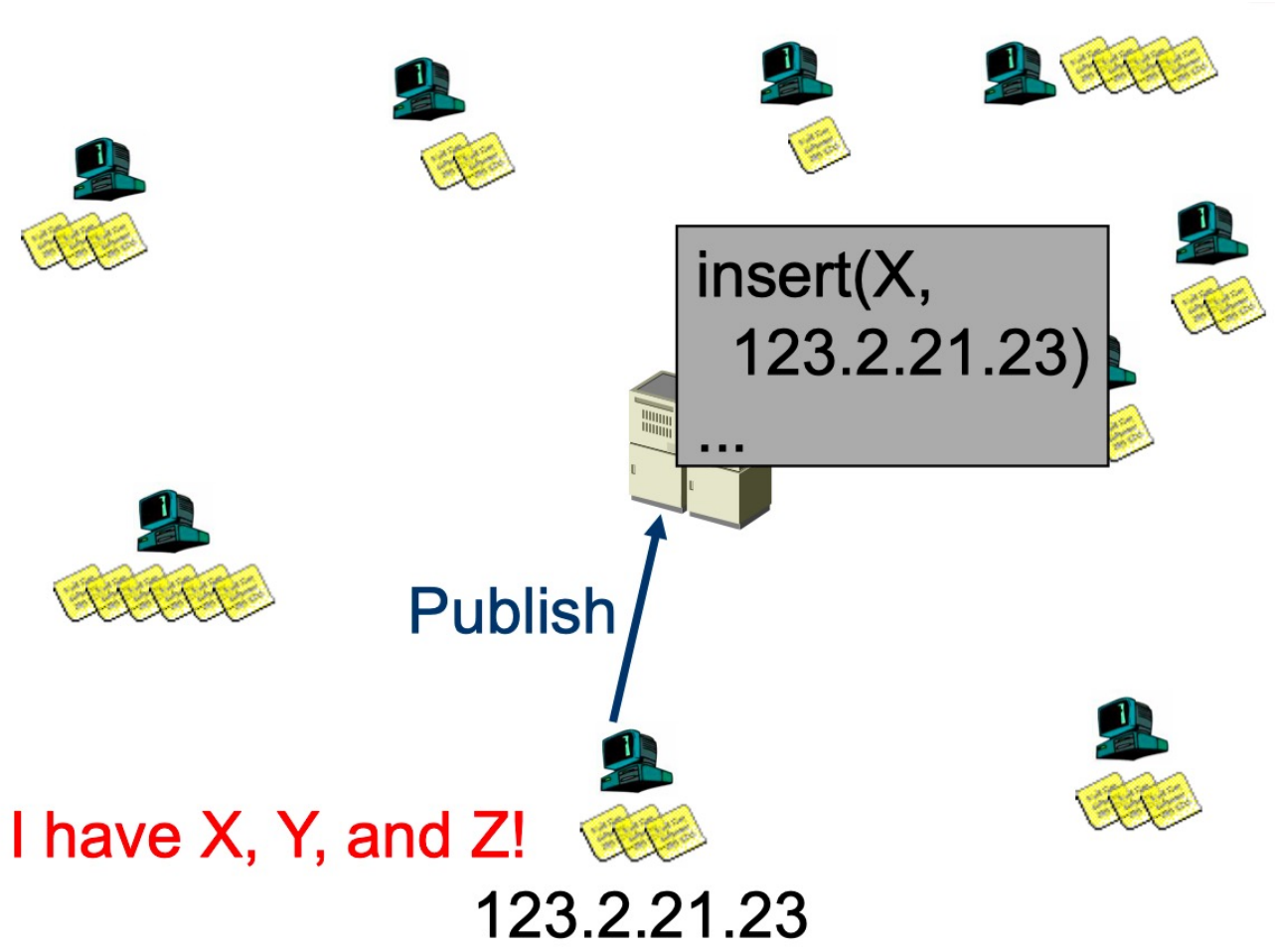
- Common primitives
  - Join: how do I begin participating?
  - Publish: how do I advertise my file?
  - Search: how to I find a file?
  - Fetch: how to I retrieve a file?

Lookups

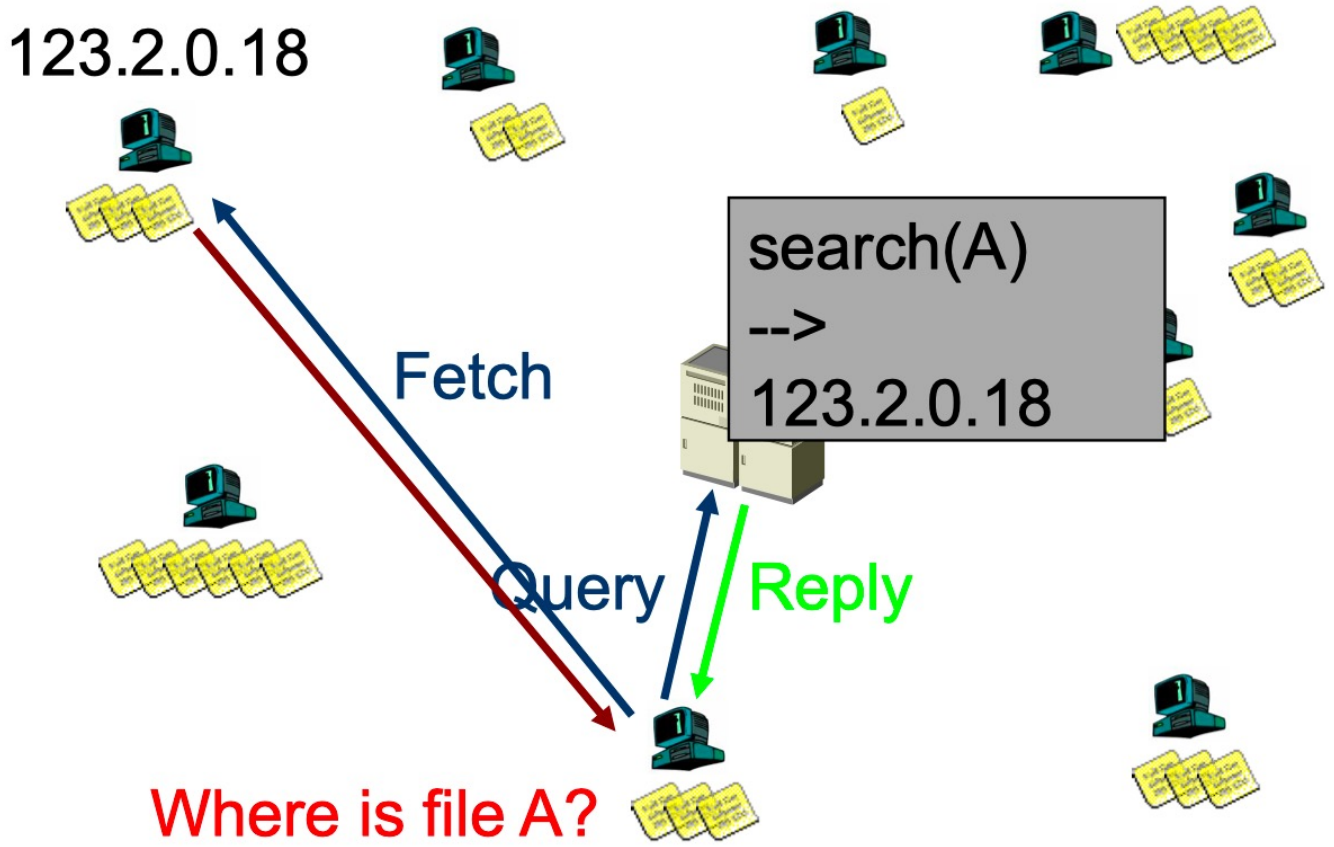
Napster

- P2P file sharing
- Centralized database:
  - Join: on startup, client contacts central server
  - Publish: reports list of files to central server
  - Search: query the server => return someone that stores the requested file
  - Fetch: get the file directly from peer

Napster: Publish



Napster: Search



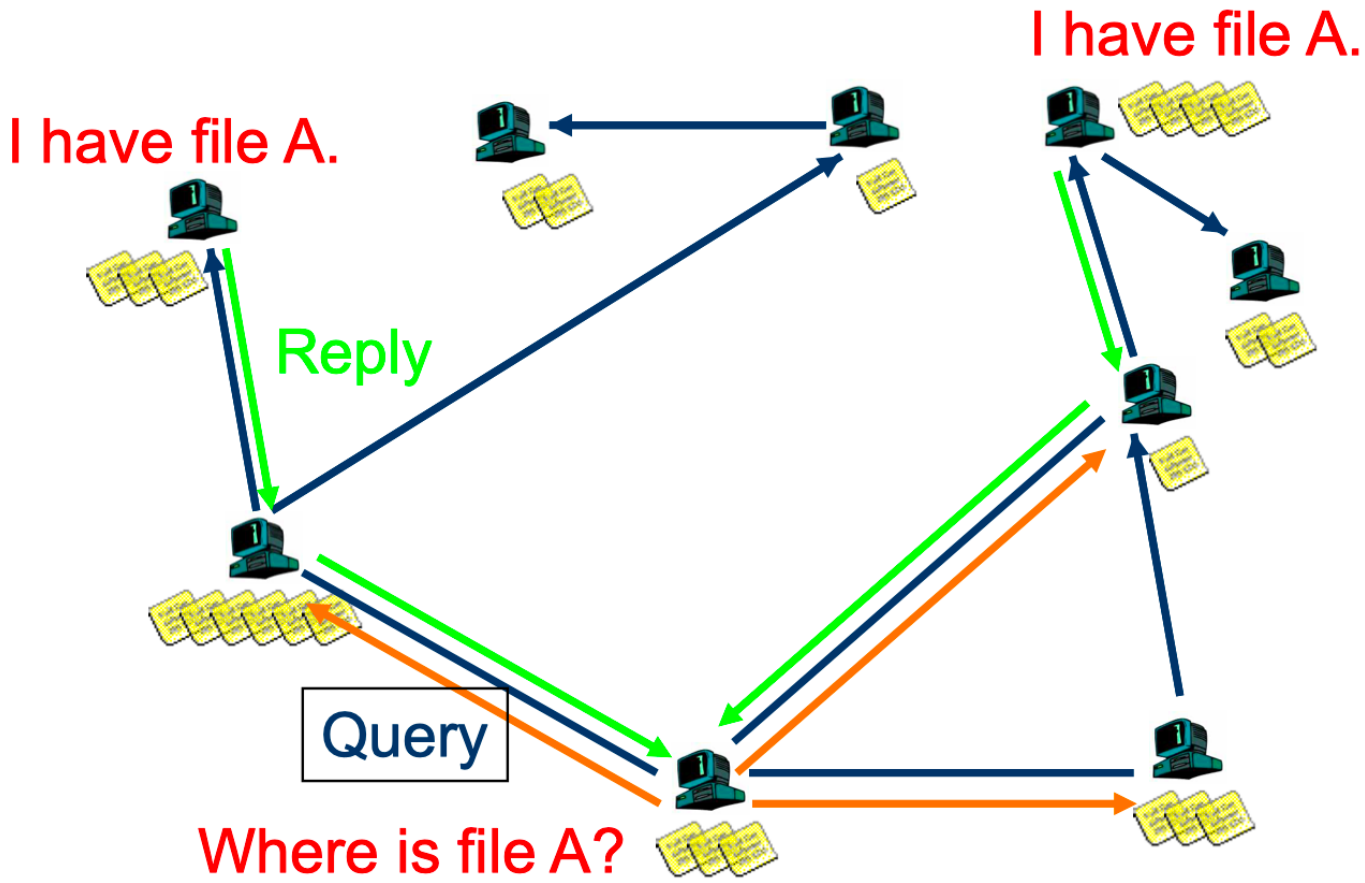
Napster

- Pros:
  - Simple
  - Search scope is O(1)
- Cons:
  - Server maintains O(N) State
  - Server does all processing
  - Single point of failure

Gnutella

- Decentralized networks for file sharing
- Query Flooding
  - Join: on startup, client contacts a few other nodes; these become its “neighbors”
  - Publish: no need
  - Search: ask neighbors, who ask their neighbors, and so on... when/if found, reply to sender.
    - TTL limits propagation
  - Fetch: get the file directly from peer

Gnutella: Search



Gnutella

- Pros:
  - Fully de-centralized
  - Search cost distributed
  - Processing at each node permits powerful search semantics
- Cons:
  - Search scope is O(N)
  - Search time is O(???)
  - Nodes leave often, network unstable
- TTL-limited search works well for haystacks.
  - For scalability, does NOT search every node. May have to re-issue query later; no guarantee that it will find the file!

Flooding: Gnutella, KaZaA

- Modifies the Gnutella protocol into two-level hierarchy
  - Hybrid of Gnutella and Napster
- Supernodes
  - Nodes that have better connection to Internet
  - Act as temporary indexing servers for other nodes
  - Help improve the stability of the network
- Standard nodes
  - Connect to supernodes and report list of files
  - Allows slower nodes to participate
- Search
  - Broadcast (Gnutella-style) search across supernodes

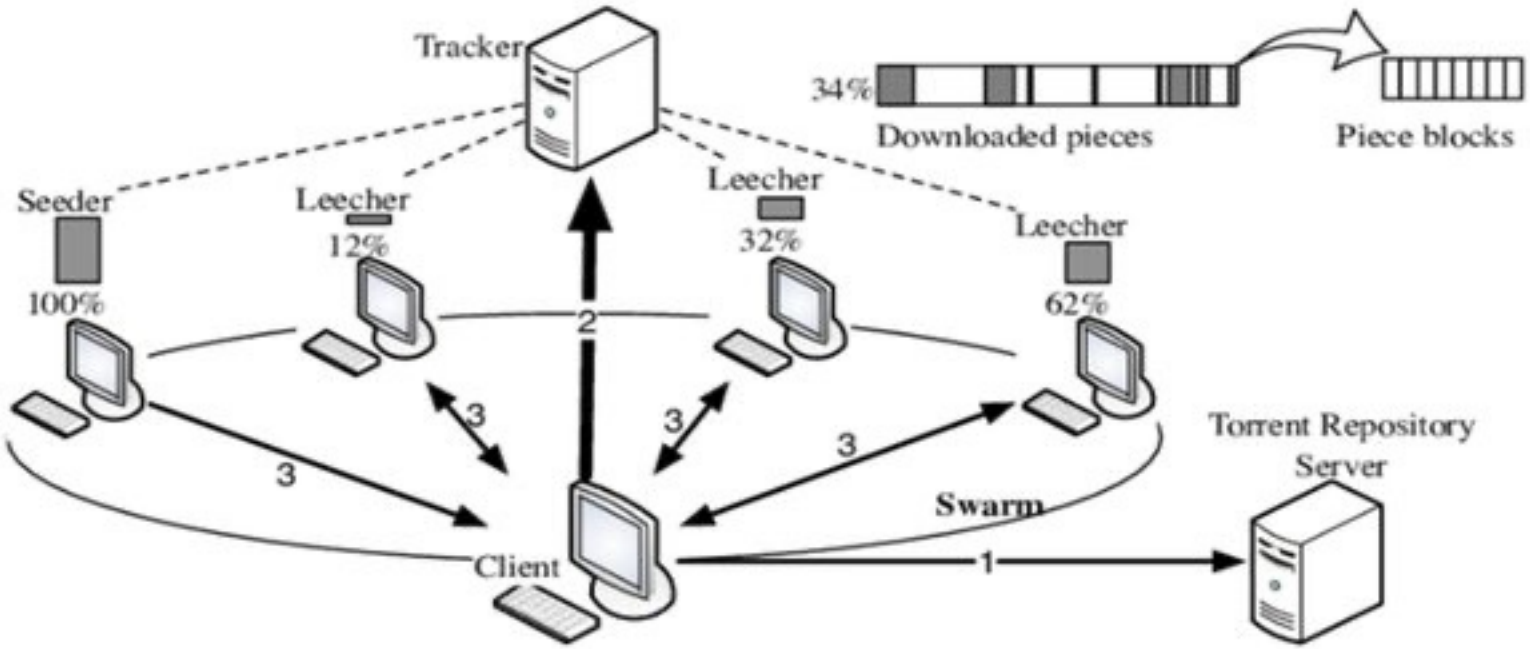
Disadvantages: Kept a centralized registration -> allowed for law suits

BitTorrent

BitTorrent

- File swarming
  - Join: contact centralized “tracker” server, get a list of peers.
  - Publish: Run a tracker server.
  - Search: Out-of-band. E.g., use Google to find a tracker for the file you want.
  - Fetch: Download chunks of the file from your peers. Upload chunks you have to them.
- Big differences from Napster
  - Chunk based downloading
  - “few large files” focus
  - Anti-freeloading mechanisms
  - Out-of-band with search engines scalable and resilient

BitTorrent


Tracker
34%
Downloaded pieces
Piece blocks
Seeder
100%
Leecher
12%
Leecher
32%
Leecher
62%
2
3
3
3
3
Torrent Repository
Server
Swarm
Client
1

BitTorrent sharing strategy

- Employ “Tit-for-tat” sharing strategy
  - A is downloading from some other people
  - A will let the fastest N of those download from it
- Be optimistic: occasionally let freeloaders download
  - Optimistic unchoke (randomly unchoke peers)
  - Otherwise no one would ever start!
  - Also allows you to discover better peers to download from when they reciprocate
- Rarest first policy: distribute rare blocks first

BitTorrent

- Pros
  - Works reasonably well in practice
  - Gives peers incentive to share resources; avoids freeloaders
- Cons
  - Central tracker server needed to bootstrap swarm
    - Alternate tracker designs exist (e.g., DHT-based trackers)

Distributed Hash Table (DHT)

DHT

- Goal: make sure that an item (file) identified is always found in a reasonable # of steps
- Abstraction: a distributed hash-table (DHT) data structure

```
key = hash(data)
lookup(key) → IP addr (Chord lookup service)
put(IP address, key, data)
get(IP address, key) → data
```

- Implementation: nodes in system form a distributed data structure
  - Can be Ring, Tree, Hypercube, Skip List, Butterfly Network, ...

DHT is expected to be

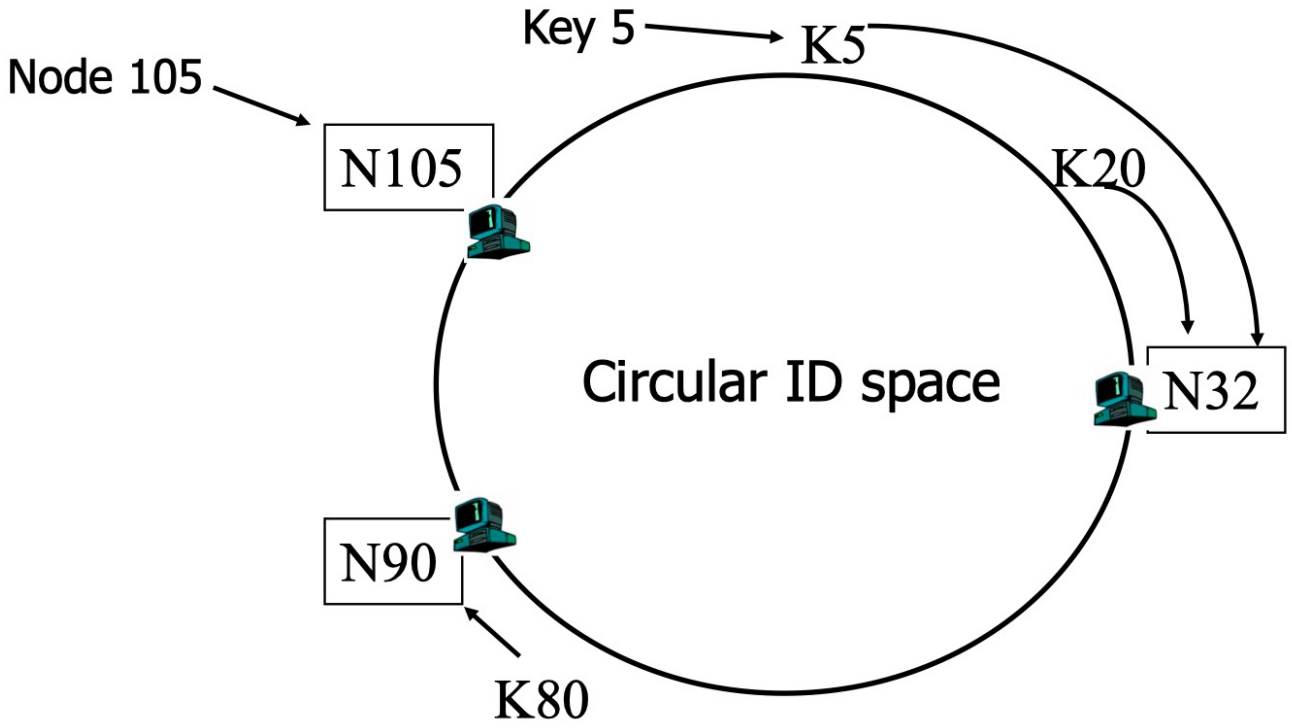
- Decentralized: no central authority
- Scalable: low network traffic overhead
- Efficient: find items quickly (latency)
- Dynamic: nodes fail, new nodes join

DHT: Consistent hashing

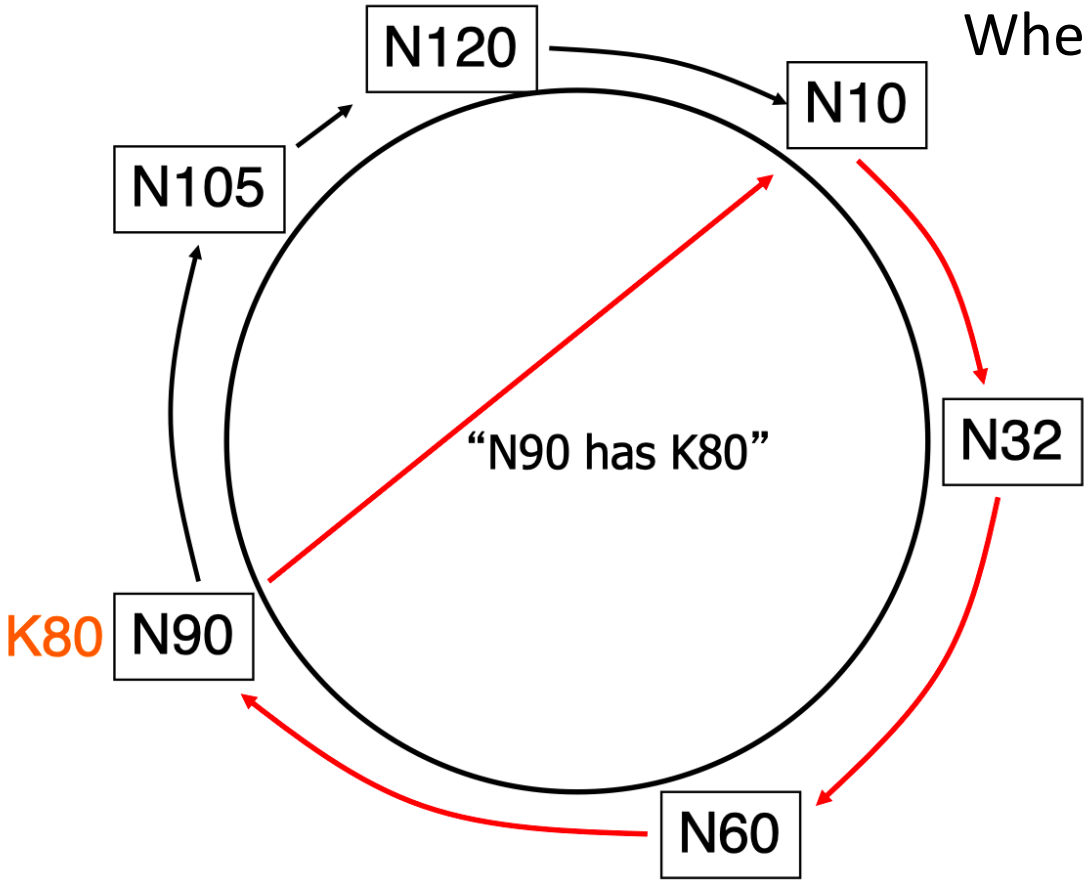
- Hashed values (integers) using the same hash function
  - Key identifier = SHA-1(key)
  - Node identifier = SHA-1(IP address)
- How does Chord partition data?
  - i.e., map key IDs to node IDs
- Why hash key and address?
  - Uniformly distributed in the ID space
  - Hashed key → load balancing; hashed address → independent failure

DHT: Consistent hashing

- A key is stored at its successor: node with next higher ID



Chord basic lookup

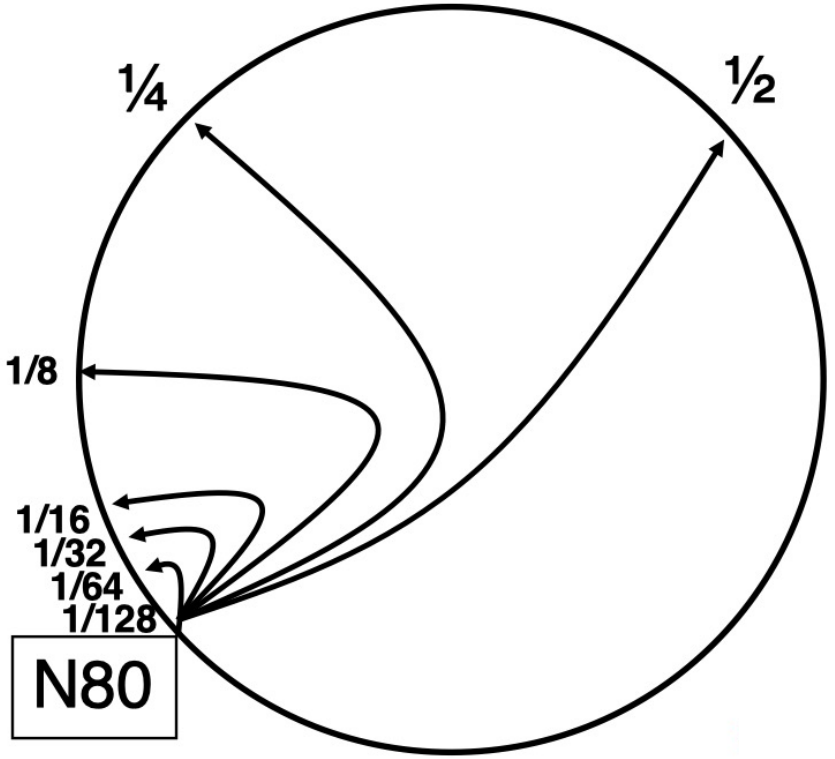


Where is key 80?

O(N)!

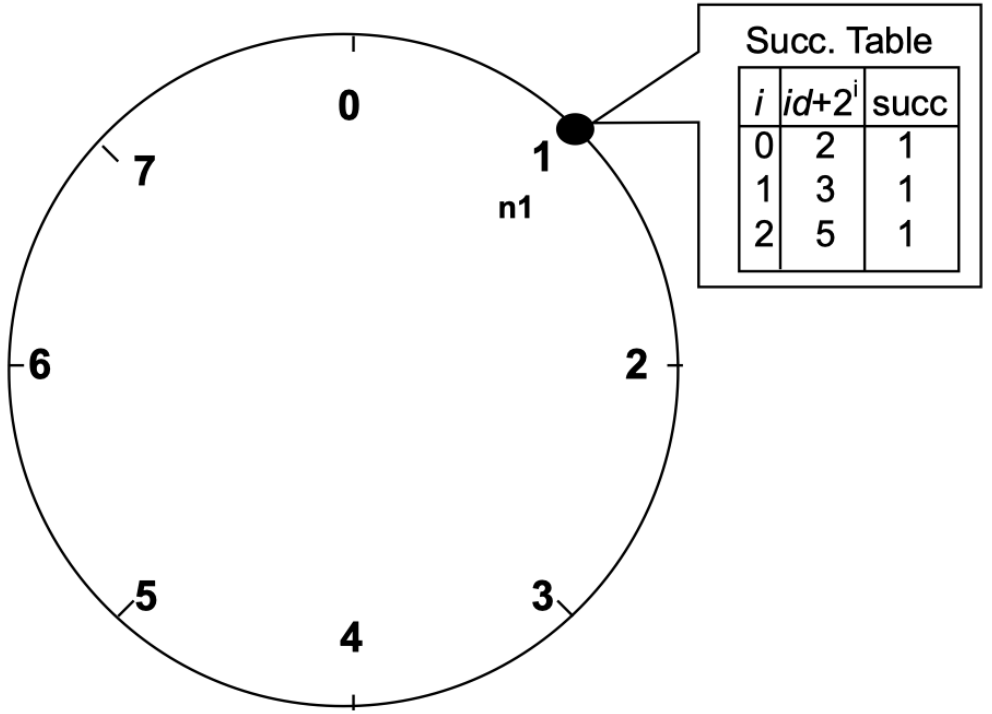
Chord finger tables (fast lookup)

- Assume identifier space is $0…2^m-1$
- Each node maintains
  - Finger table
    - Entry i in the finger table of n is the first node that succeeds or equals $n + 2^i$
  - Predecessor node
- An item identified by id is stored on the successor node of id



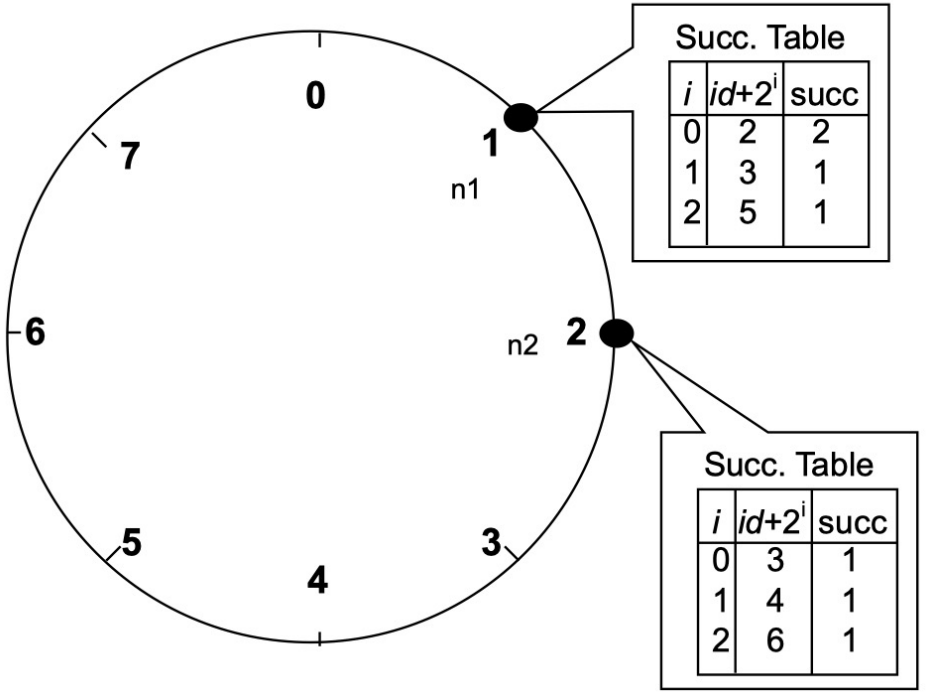
Chord example

- Assume an identifier space 0..7
- Node n1:(1) joins->all entries in its finger table are initialized to itself



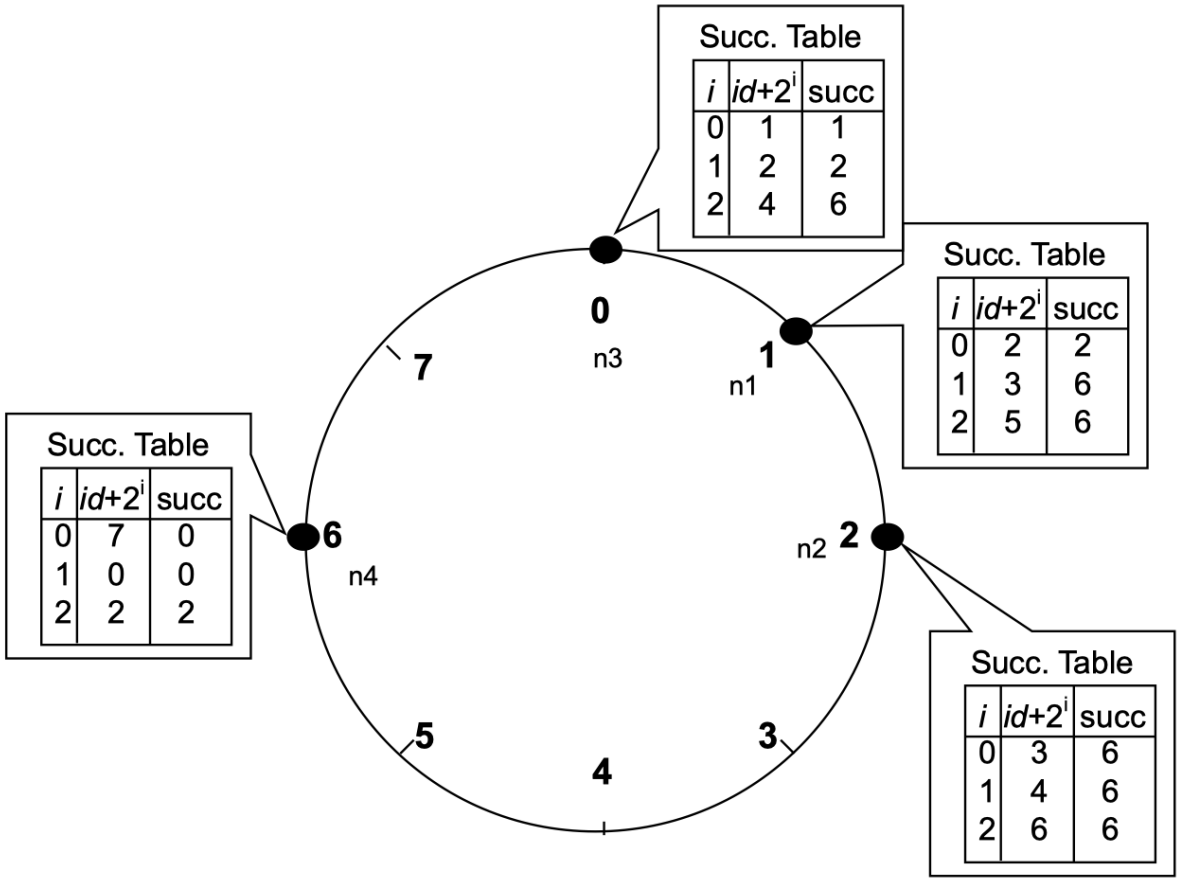
Chord example

- Node n2(2) joins



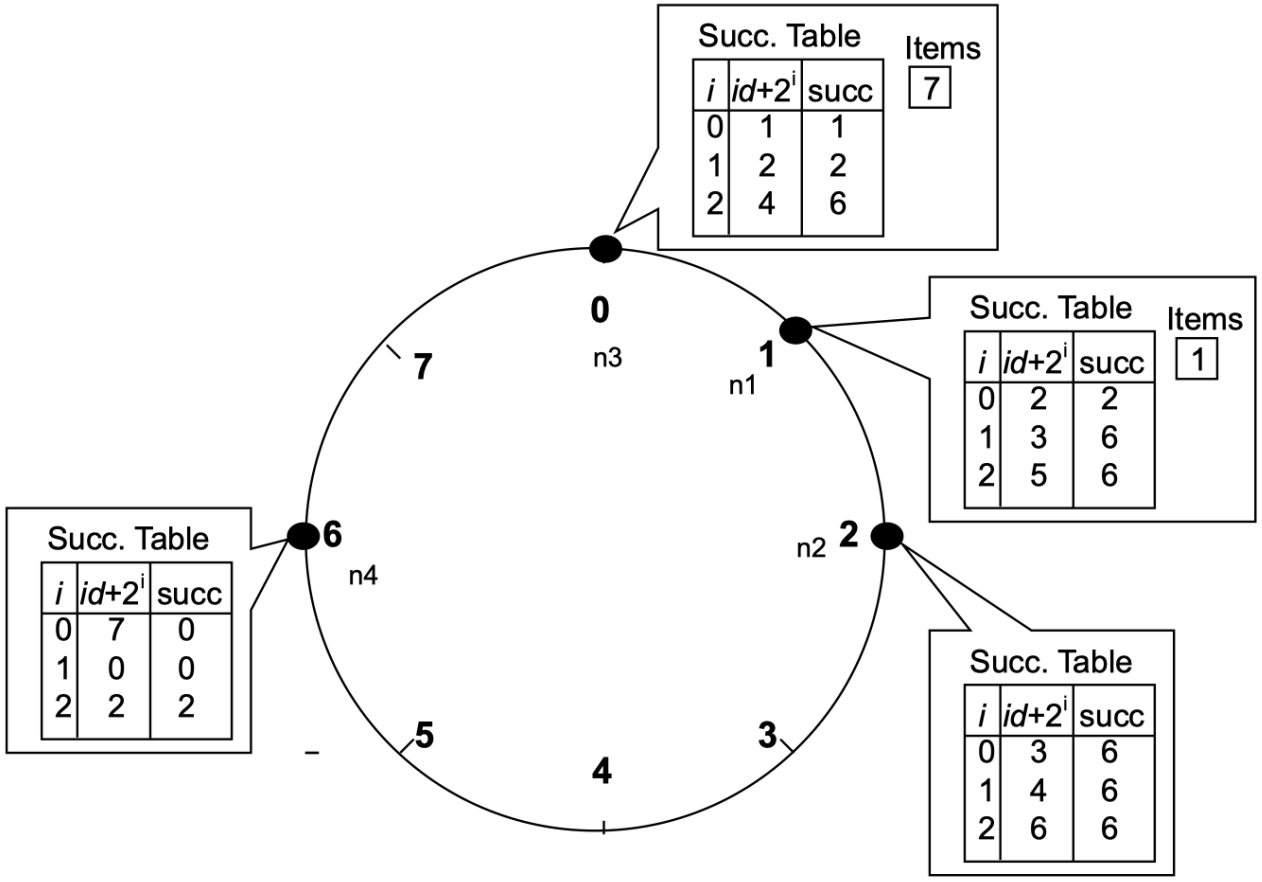
Chord example

- Nodes n3:(0), n4:(6) join



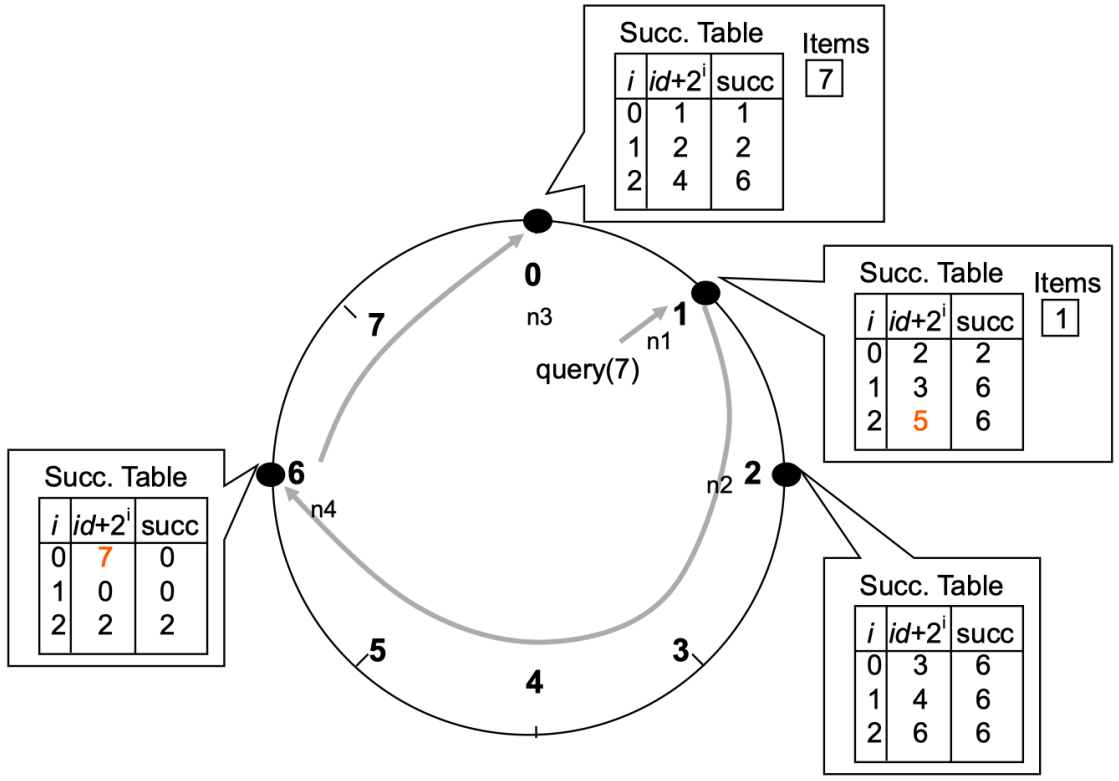
Chord example

- Nodes: n1:(1), n2(2), n3(0), n4(6)
- Items: file1:(7), file2:(1)



Chord example

- Upon receiving a query for item id, a node
  - Check whether stores the item locally
  - If not, forwards the query to the largest node in its successor table that does not exceed id



DHT/Chord summary

- Routing table size?
  - Log N fingers
- Routing time?
  - Each hop expects to 1/2 the distance to the desired id => expect O(log N) hops.
- Pros:
  - Guaranteed Lookup
  - O(log N) per node state and search scope
  - Influenced many future systems; esp. key-value stores
- Cons:
  - Supporting non-exact match search is hard

What DHTs got right

- Consistent hashing
  - Elegant way to divide a workload across machines
  - Very useful in clusters: actively used in Amazon Dynamo and other systems
- Replication for high availability, efficient recovery
- Incremental scalability
  - Peers join with capacity, CPU, network, etc.
- Self-management: minimal configuration

When are P2P/DHTs useful?

- File sharing and distribution
- Decentralized applications
  - Lookup services
  - Decentralized data storage
  - Cryptocurrency networks

When are P2P/DHTs NOT suitable?

- Centralized control needed
- Consistency requirements
- High performance required

Expensive P2P replication/fault tolerance!

Credits

- Some slides are adapted from course slides of COS 418/518 in Princeton